Agent Metabolism and Conditional Immunity: A Bio-Inspired Runtime Architecture for Reliable LLM Agents

liaozhizhi (廖至智)
Independent
liaozhizhi@foxmail.com

***Abstract*—Large language models are deployed as agents whose reports of success are taken at face value. Such an agent has no notion of what a call costs, no memory of having been wrong, and no defence that survives a restart: a model that failed twice fails for the first time again. We present Agent Metabolism OS: it gives a stateless model the two regulatory systems a cell has and a prompt does not. A hierarchy of seven layers — six in the runtime, the seventh a saturation brake delegated to an immune sidecar — makes compute a finite charge and picks between two thinking beats from complexity, cost, and trust. A penalty ledger separates accumulated damage from remaining trust, so damage outranks supply. A conditional immune response prices evidence per call: two `stat` calls for a declared low-risk append, a full `git` change set when risk is high or the system is inflamed. The immune state survives SIGKILL; a fully charged system is stopped by accumulated damage alone. Beyond runtime files, the same discipline extends outward: digest-sealed YAML rule packages let an external expert distribute policy as an interface, and an audit of the Lean 4 formal proofs released by OpenAI and DeepMind separated 80 files that elaborate without a hole from 14 that compile with exit code zero yet contain holes.**

***Index Terms*—LLM agents, runtime verification, false completion, resource-aware inference, bio-inspired computing, trust and reputation**

I. Introduction

An LLM agent is usually described as a reasoning engine. It behaves more like a CPU with three things missing: nociception, an immune system, and a bill. It does not feel a failed call, it does not remember having been caught lying, and it does not pay for the tokens it spends. The metaphor is uncomfortable because it is not an insult — a CPU is not supposed to feel pain. Something else in the system is supposed to notice on its behalf, and in current agent stacks nothing does.

Two failures follow directly. The first is *false completion*. A tool wrapper that catches an exception and returns `success` leaves the model with a perfectly consistent world in which the work is done; it reports `Task Completed`, and every later step is built on a file that does not exist. The model is not lying in any sense it can detect — it was told. The second is *cascade*. A network timeout or an unbounded retry loop consumes tokens with no notion of budget, and because nothing tracks what a call cost, the first sign of trouble is the invoice.

The prevailing responses address neither. Scaling the model improves the quality of each step but not the observation channel: a more capable model still reasons over a report it cannot verify. The second response, LLM-as-a-judge, is closer but structurally compromised — it asks a model to audit a model, reading the same narration into which the failure was written. A judge that consumes the defendant's account of events is not an independent witness, however large it is. What is missing is not more intelligence at the point of judgement, but a different source of evidence.

We therefore treat the agent as a digital organism rather than a pipeline, and give it the two regulatory systems that keep a cell alive under damage. The first is *metabolism*: seven regulatory layers (L1–L7) that make compute a finite charge to be paid before work begins, choose between a fast and an expensive way to think, decide how much of the system runs at once, and confine a failure to the compartment that caused it. The second is *conditional immunity*: an out-of-process auditor that reads the filesystem and the repository instead of the agent's report, prices evidence per call from declared risk (two `stat` calls for a low-risk append, a full change set when risk is high or the system has already been contradicted), and keeps its suspicion across the death of the process that accumulated it. Between them sits a *penalty ledger* that separates accumulated damage from remaining trust, so that a system with a full battery and a poor record is stopped by the record.

Our evaluation is deliberately adversarial rather than merely favourable. A real subprocess is killed with SIGKILL and restarted, and it resumes INFLAMED; an account at 100% charge is driven into EMERGENCY by accumulated damage alone; and the cheap tier is measured at a median of 5.7 ms with no `git` process per call, against 24.8 ms and three for the expensive one. We also report the case in which the cheap tier is wrong, because a witness that has never been wrong has not been tested.

This paper makes four contributions:

- a four-valued verdict lattice (PASS, FAIL, INVALID, NOT RUN) that separates "we verified it" from "we did not look", together with a witness whose evidence does not originate in the agent's report (Section III);
- a layered metabolic runtime that treats compute as a finite charge and whose cheap tier is cheaper because it does not exist, not because it is faster (Sections IV-A and IV-C);
- a conditional immune response with an inflammatory threshold and durable state, whose brake is deliberately

hosted outside the system it regulates (Section IV-C);
- an adversarial cross-process evaluation in which every claim is backed by a real subprocess, a real repository, and a real signal (Section V).

Two further results test whether that arrangement generalises rather than merely holding where it was built. Section IV-D moves policy out of the machine: a rule package published by a third party is installed only after its bytes match a published digest, and the package itself must cross the membrane before any of its rules may act. Section V-F points the same lattice at machine-written mathematics, where the compiler is willing to certify a file it has not proved.

II. Related Work

False completion and agent reliability. The failure this paper targets was named and instrumented in our prior work, a method-transfer audit from formal verification to agent harnesses [1]. That work establishes the vocabulary — a claim, a witness, and a four-valued verdict — and demonstrates the audit against a reference implementation; the present paper asks a different question, which is what a runtime should *do* with such verdicts once it has them. The two are complementary: the audit can name a false completion, but naming one changes nothing about the agent's behaviour unless something downstream spends a budget or withholds trust.

The obvious substitute for an independent witness is a second model. The model-as-judge literature is the place to look for its limits. Judge agreement with humans is real — strong judges reach the level of human–human agreement on open-ended comparison [2] — but the same work documents position, verbosity, and self-enhancement biases, and the self-enhancement finding has since been given a mechanism: evaluators recognise their own generations above chance, and the strength of self-preference tracks the strength of that recognition under controlled fine-tuning [3]. For our purposes the relevant consequence is structural rather than quantitative. A judge, however strong, reads the same narration the failure was written into, so improving the judge raises the quality of the opinion without changing its provenance. We therefore treat self-report and judge opinion as the same class of evidence, and collect a witness from outside that class.

Agent runtimes and orchestration. Contemporary agent frameworks are, in the main, orchestration layers. LangChain composes model calls, tools, and data sources into pipelines [4]; AutoGen organises multiple conversable agents and the patterns by which they converse [5]; LangGraph makes the control flow an explicit graph with cycles and persistent state [6]; CrewAI offers a higher-level, role-based multi-agent abstraction [7]. These systems differ in what they make easy — topology, message passing, state machines, team structure — and all of them are good at it.

What they have in common is the layer they operate at. They decide *who talks to whom and in what order*, not *what a call cost and whether it happened*. Token spend appears as an after-the-fact metric rather than a budget that can refuse a call, so a retry loop is bounded by patience rather than by a charge that runs out. A tool's report of success is passed along as the truth about the world, because the framework has no second channel to consult. And nothing in these systems remembers a contradiction: restart the orchestrator and the process begins again with the same confidence it had before it was wrong. Our contribution is not a better topology; it is the accounting and the evidence layer these frameworks leave to the operator.

Fault isolation and bio-inspired structure. The mechanisms we borrow are not new, and we would rather place them than claim them. Compartmental failure containment is the Actor model's founding move: units that share no state and communicate only by message cannot corrupt each other, and a failure stays inside the unit that failed [8]. Our L7 is a deliberately modest version of that idea, with an explicit lysis threshold and an honest note that the host language cannot enforce the membrane. The graded response to accumulating errors — trip a threshold, stop trusting the path, recover slowly — descends from the stability patterns catalogued for production systems, circuit breaker and bulkhead among them [9].

What we have not found in that lineage is the pairing. Circuit breakers protect a *callee* from a caller's retries and are driven by transport failures; they do not ask whether a call that reported success actually changed anything. Actors isolate state but do not audit effects. Our claim is that the two belong together and that the coupling should run in one direction: a metabolic layer decides what may run, an immune layer decides what may be believed, and the second one's verdicts charge the first one's ledger until damage, not supply, is what stops the system. Three details of that arrangement are, as far as we can tell, not present in the systems above: evidence is priced per call rather than per policy, the suspicion state is durable across process death, and the defensive brake is deliberately hosted outside the component it regulates. Section V measures each of them rather than asserting them.

III. Taxonomy of False Completion

We separate two things a harness usually conflates: the *claim* and the *witness*. The claim is what the agent was told (`reported_success`). The witness is what an independent observer reads afterwards: file existence and mtime, or the repository change set. A verdict is the relation between them.

Table I states the lattice. The design choice worth defending is that NOT RUN is not a fifth shade of green but a refusal to count: a run of skipped calls must not heal an inflamed system, because nothing was checked.

Why a boolean is fatal here. Conventional test suites get to be two-valued because they are *recomputable*: the assertion is a pure function of the world, and running it again reproduces the same truth. An agent's work is not recomputable in that sense. Its outcome is a side effect, and re-running the side effect to check it is not an observation but a second mutation — writing the file twice is not the same act as reading it once. The verdict must therefore be assembled from evidence captured around the call, and the value of that evidence must be recorded rather than assumed.

TABLE I

THE FOUR-VALUED VERDICT LATTICE. THE TWO MIDDLE VALUES ARE THE ONES A BOOLEAN WOULD COLLAPSE, AND COLLAPSING THEM IS HOW AN UNWITNESSED CALL BECOMES A GREEN LIGHT.

Verdict	Meaning	Why it is distinct
PASS	Claim agrees with witness.	The only value that authorises anything.
FAIL	Claim says success; witness says no effect.	A false completion: the lie is observable, so the audit can name it.
INVALID	The scenario ran, but no witness could be collected.	"We did not look" is not "we looked and it was fine."
NOT RUN	The scenario never executed.	No evidence either way; must neither heal nor inflame the immune state.

Collapse the four cases into two and the two values a boolean has no slot for are both misfiled, in opposite directions. A missing witness becomes a pass, which is the more dangerous error: the call is now indistinguishable from one that was checked and found correct, so every later consumer inherits the claim. A skipped call becomes a failure, which is not merely noise. In a runtime where failures accumulate damage, a false alarm spends trust, and an operator who learns that the audit cries wolf will route around it. Both misfilings are failures of the *vocabulary*, not of the checker.

The order. The four values are not four shades of one judgement; they differ in what they license. PASS is the only value that authorises anything and the only one that can heal an inflamed system. FAIL is damage. INVALID is also damage, and more urgent damage, because it means the witness layer itself failed while the call proceeded unchecked — which is why one INVALID trips the threshold where two FAILs are required. NOT RUN sits outside the balance entirely: it counts as neither health nor harm, because a call that never executed produced no evidence to weigh.

We express this as an ordering in which PASS is maximal for authorisation and NOT RUN is minimal for information, with the composition rule that a composite action inherits the weakest verdict among its steps. In the implementation that rule is not a join operator over verdicts — there is none — but a consequence of recording every step against the same immune state. Nine passes followed by one INVALID leave the system inflamed, because the invalid is weighed on its own terms; a long run cannot launder an unwitnessed step by surrounding it with verified ones. The per-case decision is realised as a fixed precedence, evaluated first match wins, in the order INVALID, NOT RUN, FAIL, PASS: an operational form of the same ordering rather than a separate algebra.

The ordering matters in exactly one ambiguous case. When neither an intent nor a witness exists, both NOT RUN and INVALID are defensible, and the implementation returns INVALID because that predicate is tested first. The choice is deliberate: NOT RUN is inert by design, so if silence resolved to NOT RUN a system could avoid inflammation simply by not recording what it intended to do. Ambiguity resolves toward the reading that leaves the system more suspicious.

TABLE II

METABOLIC LAYERS. L5 IS NOT A MODULE OF THIS PACKAGE: THE SATURATION BRAKE IS HOSTED BY THE IMMUNE SIDECAR, BECAUSE THE EVIDENCE NEEDED TO TRIP IT IS THE AUDITOR'S, NOT THE SCHEDULER'S (SECTION IV-C). KEEPING THE LAYER IN THE NUMBERING MARKS THE SEAM BETWEEN THE TWO SYSTEMS INSTEAD OF HIDING IT.

Layer	Mechanism	What it decides
L1	Energy account	A finite charge. Below 30% the expensive beat is forbidden; below 10% the account funds nothing.
L2	Dual-beat scheduler	OXPHOS (1× energy) or SLP (5× energy, 2× latency).
L3	Regime switch	How much runs: GROW (5 agents), CONSERVE (1), EMERGENCY (interrupt, no new writes).
L4	Substrate channel	Agents trade keys, not payloads.
L5	Saturation brake	*Hosted externally* by the immune sidecar (Section IV-C): past a contradiction threshold, cheap evidence is withdrawn and every call pays the full witness price.
L6	Circadian pass	Night mode compresses the day into content-addressed memory; read-only by design.
L7	Compartment	Membrane-bounded units; a failing unit is lysed and its neighbours keep their memory.
—	Penalty ledger	Accumulated damage, read by L2 and L3 (Section IV-B).

An independent witness is the precondition for PASS. The engine tests for a missing witness before it reads the agent's report at all. A confident self-report accordingly cannot buy a PASS; the best it can do with no evidence behind it is produce INVALID, and the reason string records that refusal explicitly rather than leaving it implicit. This ordering is the whole design in one line. A checker that reaches for the claim first and the evidence second will occasionally award a PASS on the strength of the claim, and from that moment the verdict carries no information the agent did not supply itself.

That is what we mean by the beginning of corruption, and it is not a moral claim. A witness-free PASS does not merely produce one wrong value; it destroys the only channel through which the error could later be discovered, so no downstream mechanism — not a larger judge model, not a longer review — can recover what was never observed. Keeping the two apart is cheap while it lasts and impossible to restore afterwards.

IV. THE BIO-INSPIRED ARCHITECTURE

A. Metabolic Layers (L1–L7)

The model is treated as a CPU with no memory of its own charge: ask for a long chain of thought on the last joule and it will produce one. Regulation therefore lives outside it. Table II lists the layers and where each one is implemented.

L1 and L3 use different thresholds (30/10 versus 70/30) on purpose: one governs whether a single call may think slowly,

the other how many calls may run at once. An account at 65% is legitimately GROW for L1 and CONSERVE for L3.

L5 is the one layer that deliberately lives outside this package. A saturation brake is only worth having if it can be tripped by evidence the regulated system does not produce: a brake that reads the regulator's own accounting can be tripped by the same defect that corrupts the accounting. We therefore place it where the contradiction is observed, in the immune sidecar. In that implementation, tripping L5 withdraws the cheap witness from every call rather than denying permission; denial is L3's write gate, and it is reached through the ledger (Section IV-B). The split is this paper's thesis in miniature: the metabolic system decides what may run, the immune system decides what may be believed, and what the second one sees is what eventually stops the first.

L4: a handover costs a key, not a payload. The channel's unit of transfer is a short key, not the object. An agent that publishes a 40 KB document calls `put` and receives an identifier; the membrane message that carries it is the string `substrate:spec_v1`, thirteen bytes regardless of what was published. The recipient resolves the key through `get`, which returns the live object rather than a copy. We make the shared view the default deliberately: a channel that deep-copied on every transfer would reintroduce, quietly and per call, the exact cost it exists to remove. Isolation is not this layer's job — it is L7's — and conflating the two turns a transport optimisation into a correctness bug in both directions.

Two properties keep that default from becoming a hazard. First, a live key cannot be reused: publishing under an existing key raises rather than overwriting, because an in-place overwrite would silently change what every holder is about to read — an invisible corruption that no participant would report. Second, copying is available but must be requested: `get_snapshot` and `put(copy_data=True)` hand a caller a private copy when sender and reader must not observe each other's mutations. The asymmetry is the invariant. The fast path is shared by default, and the isolation cost is paid only by the callers that declare they need it.

L6: consolidation reads and never rewrites. Night mode walks the live compartments, reads each one's admitted context through its public view, and appends a content-addressed fragment to a long-term file. It has no write path back into any compartment; the sources of a consolidation pass are unchanged after it. The reason is adversarial rather than aesthetic. A routine that pruned its own inputs would delete, on a schedule, the audit trail a later investigation depends on, and with nobody having asked for a deletion. If memory pressure ever requires pruning, that operation belongs to L7's lysis, where it is explicit, scoped to one compartment, and recorded with a reason.

Consolidation is also idempotent by construction: a fragment's identifier is a digest over the exact messages it summarises, so a second pass over an unchanged day yields the same identifiers and appends nothing. Content addressing does the work that a "have I already summarised this?" flag would otherwise have to do, and it cannot fall out of sync with its subject, because it is derived from the subject. We state the limitation plainly: the pass is a structural digest — it deduplicates, counts, and extracts recurring terms — not a semantic summary, and the stored record is labelled accordingly, so a later reader knows which kind of memory they are reading.

L7: lysis is local. A compartment exposes its context as a tuple and has no method that writes another compartment's memory. The only cross-compartment path is a manager-mediated delivery into the recipient's inbox, which the recipient alone admits into its context on its own schedule; a sender can therefore put a message in front of a peer but cannot edit what that peer has already admitted. Isolation is structural rather than advisory, and it is what makes containment possible at all.

When a compartment fails repeatedly — the implementation lyses after three consecutive anomalies, a run broken by any success — its context and inbox are cleared, its status becomes LYSED, and it stops routing. Nothing else is touched: every other compartment keeps its context, its inbox, and its routes. We report this as the property under test rather than as an intention, and note the limitation honestly. The host language provides no access control, so the boundary is an immutable public view plus a single private delivery path, not a hard security boundary. The claim is the containment of failure, and it is enforced exactly where the code can enforce it.

B. The Penalty Ledger

Every other layer measures *supply*. None measures *damage*: a system can sit at 100% charge with a perfect error window and still have shipped three false completions. The ledger is that memory, and it keeps two units that refuse to move together.

For a charged violation with severity weight w and base B,

$$P \leftarrow P + wB, \tag{1}$$

$$T \leftarrow \max\bigl(0,\ T - \tfrac{1}{2}wB\bigr)\,, \tag{2}$$

$$T \leftarrow \min(100,\ T + 2) \quad \text{per verified pass,} \tag{3}$$

$$P \leftarrow 0.98\,P \quad \text{per verified pass.} \tag{4}$$

Damage decays geometrically while trust recovers linearly, and the asymmetry is deliberate: a fine can be paid off, a broken claim cannot be paid back by bookkeeping.

Two meters, and why they are two. The ledger maintains a fine and a trust score as separate quantities, and the separation is the design rather than an implementation detail. A single scalar would have to answer a question that has no single answer: when is a catastrophe *paid off*? If the scalar is the fine, then a system that has amortised its debt presents as healthy while the damage it did is still unrepaired. If the scalar is trust, then the record of what happened disappears the moment confidence returns. Keeping both means the ledger can show the uncomfortable state the two meters are for: trust fully bought back while the fine still classifies the system as damaged.

The recovery rates make that state unavoidable. A verified pass buys back a fixed 2 points of trust but retires only 2% of the outstanding fine, so a large fine keeps hurting long after confidence has returned. Starting from the three false completions charged in Section V (fine 150, trust 25), the trust

TABLE III
DEFAULT SEVERITY WEIGHTS. THE FINE IS wB WITH $B = 10$, SO A FALSE COMPLETION COSTS 50 UNITS AND 25 POINTS OF TRUST. AN UNLISTED TYPE MUST SUPPLY w EXPLICITLY; IT DOES NOT INHERIT ONE.

Violation type	w	**Fine at** $B = 10$
FALSE COMPLETION	5.0	50.0
ISOLATION BREACH	4.0	40.0
UNAUTHORISED ACTION	4.0	40.0
UNWITNESSED WRITE	3.0	30.0
BUDGET OVERRUN	2.0	20.0
PROTOCOL VIOLATION	1.5	15.0
STALE SUBSTRATE	1.0	10.0
unlisted	*required*	*must be stated*

score is whole again at the 38th consecutive verified pass; the fine does not fall below the critical line until the 66th, and the ledger does not read HEALTHY until the 114th. We report the trajectory rather than a single recovery time because the gap between the two meters *is* the claim.

The weights themselves are policy, not measurement, and we say so rather than dressing them as empirically derived. What the table fixes is the ordering and the cheapest possible confession of a novel failure: a type that is not listed must state its own weight, because charging an unknown failure at some default is how a new and unfamiliar defect gets under-reported.

Table III places false completion at the top, above isolation breach, and the ranking is intentional: an isolation breach is loud and is caught by the layer that owns it, whereas a false completion is silent, survives review, and is discovered later by someone else. The ledger prices silence higher than noise.

Boundaries lean toward the worse side. The asymmetry with L1 is deliberate and visible in the code that both are computed from. L1 treats exactly 30% charge as GROW, because charge is a resource being consumed and standing on the line is not yet a problem. The ledger treats exactly 40.0 as CRITICAL, because damage is not a resource and a threshold rounded in the system's own favour is how a ledger stops being a ledger. The same convention applies to trust: 80.0 is already WARNING and 50.0 is already CRITICAL. One direction protects the system's ability to work; the other protects the operator's ability to believe it.

C. Conditional Immunity

The audit can be made cheap or it can be made complete, and paying the complete price on every call is how a runtime audit gets switched off in production. We therefore price evidence per call, from three inputs:

1) the rule's declared `risk_level`;
2) whether the call names a path that can be observed at all;
3) the current immune state.

Tier 1 (innate). Two `stat` calls per claimed path — existence, `st_mtime_ns`, size — taken before and after the command. No git process, no file read. Sufficient for a declared low-risk append, where the question is only whether the append landed.

Tier 2 (adaptive). The full repository change set plus a content read. Used when risk is declared high, when a rule demands a witness, and always when the system is inflamed.

Two asymmetries carry the design. First, *a tool with no rule at all is judged on Tier 2*: absence of a policy is not evidence of low risk, and the opposite default would mean that adding a rules directory could make the audit weaker for every tool it forgot to mention. Second, *going cheap never loosens a verdict*: a Tier-1 false completion is still FAIL, and every record carries its tier and witness scope, so a PASS on an mtime can never be read as a PASS on a change set.

The inflammatory storm. Two consecutive FAILs, or a single INVALID, set the state to INFLAMED; five consecutive PASSes heal it, and any contradiction in between restarts the count. While inflamed, every call is forced onto Tier 2 regardless of its own declared risk, and NOT RUN is neither counted as health nor as damage.

This is L5 in operation, so it is worth being exact about what tripping it does: it withdraws cheap evidence, it does not deny permission. Denial is L3's write gate, reached through the ledger. A brake that also arrested every call would be indistinguishable from a crash from the outside, and an actor that can induce crashes would then own the off switch.

Persistence. A state held only in memory is a jailbreak: crash the sidecar, restart it, and the suspicion is gone — and making the sidecar crash is something the audited agent can arrange. The counters are therefore written to a small JSON document after every verdict, by atomic replace, and read back at construction. A missing file is an ordinary first run; a malformed or unknown-version one is logged as an *amnesia event*, because silent forgetting and induced forgetting look identical from outside. Every ambiguity is resolved toward suspicion: a record that says INFLAMED with a zero danger count stays inflamed, and a recovery counter at or past its threshold is held one short, so a forged record can make the system more suspicious but never calmer.

The adjudication procedure. Algorithm 1 states the whole decision as it is implemented, including the three escalation conditions that are easy to omit when the mechanism is described informally. Escalation is monotone: every reason to spend more evidence wins, and one reason is sufficient. A call that names no observable path escalates rather than returning INVALID, because the Tier-1 witness would have nothing to sample and the resulting INVALID would say more about the chosen tier than about the call — and would itself inflame the system.

Residual risk. The state file is unsigned. A writer with access to that path can delete it, and deletion restores the calm of a first run. What the mechanism guarantees is that a restart no longer *cleanses* the state by itself — the specific failure in which killing the auditor is an escape hatch. It does not guarantee tamper resistance, and a deployment that needs that guarantee must either place the file where the audited process cannot write, or sign the record. We have implemented neither,

Algorithm 1 Conditional adjudication of one tool call. SELECT implements the tier policy; RECORD advances the immune state exactly once per verdict. The verdict is one of the four values of Table I.

Require: call c (tool, claimed paths, reported success)
Require: policy P, immune state S
Ensure: verdict v, updated state S

function ADJUDICATE(c, P, S)
  $tier \leftarrow$ SELECT(c, P, S)
  $W \leftarrow$ COLLECTWITNESS($c, tier$) ▷ Tier 1: `stat` before/after; Tier 2: change set
  $v \leftarrow$ VERDICT($c.reported, W$)
  $S \leftarrow$ RECORD(S, v)
  return (v, S)
end function

function SELECT(c, P, S)
  if $S.inflamed$ **then return** TIER2
  end if
  if P has no rule for $c.tool$ **then return** TIER2
  end if
  if c names no observable path **then return** TIER2
  end if
  for $rule$ **in** $P[c.tool]$ **do**
    if $rule.risk \neq$ LOW **then return** TIER2
    end if
    if $rule.require_witness$ **then return** TIER2
    end if
  end for
  return TIER1
end function

function RECORD(S, v)
  if $v =$ NOT RUN **then**
    return S ▷ no evidence either way
  end if
  if $v =$ PASS **then**
    if $S.inflamed$ **then**
      $S.recovery \leftarrow S.recovery + 1$
      if $S.recovery \geq 5$ **then**
        $S \leftarrow$ (NORMAL, 0, 0)
      end if
    else
      $S.danger \leftarrow 0$ ▷ isolated failure is a normal day
    end if
  else
    $S.danger \leftarrow S.danger + 1$
    $S.recovery \leftarrow 0$
    if $S.danger \geq \theta(v)$ **then**
      $S.inflamed \leftarrow$ TRUE ▷ θ(FAIL) $= 2$, θ(INVALID) $= 1$
    end if
  end if
  PERSIST(S) ▷ atomic replace
  return S
end function

and we prefer to say so where the mechanism is described rather than in a limitations paragraph at the end.

D. Rule as an Interface

A keyword is how a discipline hands a question to a stranger. It says nothing about the answer; it fixes what the answer is about, so that a reader who never met the author can find the work, judge it by the same standard, and disagree in the same vocabulary. The interface is not the paper. It is the part of the paper that survives leaving its author's hands.

Rules occupy that position for agents, though they are rarely treated as an interface. A YAML file in the working directory is a note to oneself: it is edited by whoever is present, read by the same process it constrains, and no one outside can tell whether the copy in force is the copy that was reviewed. Conditional immunity already refuses to let the audited system set the price of its own evidence (Section IV-C); a rule file that the audited system can edit reintroduces exactly that weakness one level up, where whoever can change the file can change the criterion. What follows is the same argument at the boundary of an organisation, between a rule written by the operator of the agent and a rule written by someone who will never run it.

A rule package is therefore required to answer three questions that a local file never had to. *Who is this?* The identifier is namespaced and dotted (`org.example.safe-web3-transfer`) and carries a version and an author, so a trace names the policy that decided rather than the file it happened to live in. *What does it need?* An `interface_spec` names the physical witnesses the rule demands, drawn from a closed vocabulary — `git_diff`, `compiler_exit_code`, `test_exit_code`, `transaction_receipt` and six others. A rule that cannot state its evidence requirement cannot be checked by anyone but its author. *What does breaking it cost?* A `penalty_weight`; a rule whose violation is free is an advertisement rather than a gate.

The seal precedes the reading. Installation verifies the SHA-256 digest of the package's exact bytes *before* the YAML is parsed. A mismatch raises and the registry is left exactly as it was: a half-installed policy is worse than none, because its gaps are invisible. There is no unsigned local mode, and the omission is deliberate — a working directory is precisely where a rule file is most likely to be edited by hand, and an edit is what the digest exists to make visible. The schema is enforced with the severity this design applies everywhere else: unknown fields, an unsupported witness name, a duplicated identifier across two packages, and a package schema the loader does not speak are refused at load time rather than ignored. A guard believed live and in fact dead is worse than no guard, and an ignored field is a guard that its author still believes in.

The word "signature" needs a qualifier here, and we would rather supply it than have it discovered. What is checked is a digest, in the sense that `sha256sum -c` checks one: a transport seal against corruption, not a claim of authorship. If the package and its digest travel over the same channel, an adversary able to replace the package can usually replace the digest beside it, and the implementation logs a warning when it is forced to take a digest from a sidecar stored next to the bytes it is meant to vouch for. Turning that seal into an authorship claim requires a publisher key and a signature scheme over the digest; we have not implemented one, and we record it as the next step rather than as a property already

held.

The package crosses the membrane before its rules may act. Mounting is not a registration in a table; it is an admission. The registry hands its installed packages to a `MembraneGateway` and constructs a rule interface, and construction is where the crossing happens: the package digest is registered for `read`, an inline warrant is requested over the package's own text, the warrant is redeemed, and the mount is appended to the hash-chained ledger as an audited action. Only then does any rule in that package decide anything. Two properties follow, and we consider neither optional. First, the membrane module is not modified and does not learn that rules exist; injection uses the public API only, so the policy layer cannot acquire a privileged corner inside the gate it configures. Second, a package that will not cross does not load. The interface is subject to the interface, which is the only arrangement under which the gate's own configuration is evidence rather than assertion.

Enforcement keeps the two layers apart on purpose. The rule layer asks whether the evidence an action needs was collected: a missing `compiler_exit_code` is a denial even when the shell command exited zero, and the denial is written to the ledger carrying the rule identifier, the missing witness names and the penalty weight. The membrane asks a different question — whether the exact bytes offered as evidence are the bytes someone registered before the request arrived — and answers with a single-use warrant. A caller may put both questions in one step by offering a payload; leaving the payload out proves only that the evidence exists, which is adequate for a dry run and insufficient for a write. Denial is also not damage: a request stopped before execution is recorded as a denial with no verdict at all, so a defence that works cannot inflame the system it protects. The distinction between a blocked call and a failed one is what lets the penalty ledger read a defence as a defence.

Why this belongs in the architecture. It would be easy to file the registry as packaging. It is structural because it moves three things out of the code and into data that can be reviewed, versioned and refuted: the criterion of completion (the required witnesses), the price of failing it (the penalty weight), and the name of the party who set it (carried into every trace). A keyword lets a field agree on what counts as a question without agreeing on the answer; a rule package lets two organisations agree on what counts as evidence without either one running the other's agent. The closed witness vocabulary is what keeps that agreement honest, since adding a witness type is a code change: somebody has to agree to collect it. An interface able to demand evidence no runner can produce is indistinguishable from one that never loads.

V. Evaluation

Setup. Every number in Sections V-A–V-E comes from a single audit run in which the auditor was a real `stdio` subprocess, the witness was a real `git` repository, and the restart was a real SIGKILL. The two unit suites passed in full (302 + 164 = 466 tests); the audit itself asserted 67 invariants and all held. No mock stands in for the auditor. The two systems are `harness-audit-sidecar` v0.7.0 [10] and `agent-metabolism-os` v0.3.0 [11] on Python 3.12.3 and git 2.43.0. The manuscript sources are kept at [1].

TABLE IV
ONE SESSION, ONE POLICY. THE TIER IS CHOSEN BEFORE ANY VERDICT IS RECORDED, SO THE LAST ROW'S TIER 2 IS A CONSEQUENCE OF THE STATE WRITTEN BY ROWS 2–3, NOT OF THE CALL ITSELF.

#	Tool	Tier	Witness	Verdict	State after
1	`append_file`	1	`stat`	PASS	NORMAL
2	`write_file`	2	full	FAIL	NORMAL
3	`write_file`	2	full	FAIL	INFLAMED
4	`append_file`	2	full	FAIL	INFLAMED

TABLE V
WARMED, SINGLE-PROCESS BENCHMARK: IDENTICAL `APPEND_FILE` CALLS, IMMUNE STATE AS THE ONLY VARIABLE. ALL TIMES IN MILLISECONDS. LATENCY INCLUDES THE JSON-RPC ROUND TRIP AND THE `BASH` SPAWN, SO THE RATIO IS AN ORDER OF MAGNITUDE, NOT A CONSTANT; THE PROCESS COUNT IS STRUCTURAL.

Configuration	n	Median	Min	Max	git/call	Verdict
Tier 1 (calm)	5	5.7	5.4	7.0	0.0	5×PASS
Tier 2 (inflamed)	4	24.8	21.8	26.4	3.0	4×PASS
Ratio						4.3× / 0 vs 3 procs

Three implementation details of the measurement matter more than the numbers: the mediator (`git`) was replaced by a shim that logs each invocation and then `exec`s the real binary, so "did this tier spawn a process" is counted rather than inferred; the benchmark ran warm, in one process, on one repository, with the immune state as the only variable; and PASS was required in every timed call so that the two tiers are compared on equal work.

A. Verdicts and Tiers

Table IV shows one session in which the tier rises without the policy changing. Call 4 is the same low-risk append as call 1; it is judged on Tier 2 only because of the two failures above it.

B. Cost of Evidence

Table V reports the same tool on the same repository under the two tiers. The saving is not that Tier 1 is a little faster; it is that Tier 1 spawns no process at all.

The raw samples are 5.7, 5.7, 5.8, 5.4, 7.0 ms for Tier 1 and 26.4, 25.5, 24.1, 21.8 ms for Tier 2. We report them rather than only a median because n is small enough that a reader should see the spread.

C. Immunity Survives the Process

Table VI is the central result. The first process is killed with SIGKILL (exit -9); a second, differently numbered process reads the state file and continues.

The post-restart call is the *same* low-risk append that a calm system judges on Tier 1. Here it was forced onto Tier 2,

TABLE VI
CROSS-PROCESS SURVIVAL. DIGESTS ARE TRUNCATED TO EIGHT HEX DIGITS AND RECOVERY_PROGRESS STAYED 0 THROUGHOUT, SO NO RESTART HEALED THE STATE. THE DECISIVE OBSERVATION IS THE TIER OF THE POST-RESTART CALL: TIER SELECTION READS THE IMMUNE STATE BEFORE ANY VERDICT, SO A PROCESS THAT HAD FAILED TO RESTORE INFLAMED COULD ONLY HAVE JUDGED A LOW-RISK APPEND ON TIER 1.

Checkpoint	PID	State	Danger	SHA-256
After two failures	76295	INFLAMED	2	`a2258cd9`
After SIGKILL	76295	INFLAMED	2	`a2258cd9`
After restarted verdict	76309	INFLAMED	3	`3a5ca012`

TABLE VII
DEGRADATION AT FULL CHARGE. ONLY CRITICAL WITHHOLDS THE SLOW BEAT, SO THE RESPONSE IS GRADED RATHER THAN BINARY: A WARNING LEDGER ALREADY FORBIDS EXPANSION BUT STILL PERMITS EXPENSIVE THINKING. ALL THREE COLUMNS HOLD THE SAME 100% CHARGE.

	A: baseline	B: warning	C: pain
Energy	100%	100%	100%
Penalty	0.0	15.0	150.0
Trust	100.0	92.5	25.0
Ledger	HEALTHY	WARNING	CRITICAL
L3 regime	GROW	CONSERVE	EMERGENCY
Concurrent agents	5	1	1
L2 beat	SLP	SLP	OXPHOS
New writes	yes	yes	**refused**

spawned three git processes, and returned FAIL. The control run — identical call, identical repository, memory-only state — stayed on Tier 1 and still returned FAIL. That control is what makes the comparison causal: the verdict did not change, only the price of reaching it.

D. Damage Outranks Supply

Table VII holds energy fixed at 100% in all three rows. The question is whether accumulated damage can stop a system that has every joule it needs.

Row C is the claim in one line. The ledger reports `trust bankruptcy stops the system with charge at 100%`; the slow beat is withheld for lack of trust, not lack of energy (`penalty_critical`); and the write gate refuses the call without touching disk. Row B shows the grading: L3 has already withdrawn expansion while L2 still allows the expensive beat.

E. The Blind Spot We Measured

A cheap tier that is never wrong would be suspicious. Using a policy supplied only for the experiment (declaring a shell command low risk), a command that correctly appended to the claimed file *and* wrote an unclaimed file was judged PASS on Tier 1 with an empty unexpected-change set. The same command on an inflamed system escalated to Tier 2, returned FAIL, and named the unclaimed file. Tier 1 is accurate within the range it declares, and it under-reports outside it. This is why the shipped policy declares overwrites high risk and why an unmentioned tool defaults to Tier 2.

F. Auditing AI Formal Proofs

Two judges, and why neither is a witness. A machine-written proof arrives with a claim attached: that the theorem is proved. Two instruments are normally offered to check it, and neither answers the question the claim raises. The first is a language model reading the proof, which is the judge of Section II in a new costume: it reads the same narration the claim was written into, and its errors are fluent. The second is the compiler, which is genuinely independent of the narration but was never asked whether the proof is complete. In Lean [12], `sorry` closes any goal and elaborates to an axiom named `sorryAx`, and `axiom` introduces a proposition its author has simply declared true. Both compile. Measured on Lean 4.33.1, `theorem t : (1 : Nat) = 1 := by sorry` exits 0 with a warning, and `axiom riemann : True` followed by `theorem u : True := riemann` exits 0 with no output at all. A gate that reads the exit code therefore certifies a file it has not proved, which is a false completion committed by the auditor.

Three witnesses and a lattice. The `LeanProofAuditor` collects three. The first is lexical, and runs over a masked copy of the source with comments and string literals blanked and nested block comments stripped, so that `sorry` in a docstring is not an accusation; character literals are skipped rather than blanked, because an apostrophe is legal inside a Lean identifier. A `sorry` or `admit` is a hole, and an `axiom` is a proposition asserted rather than proved; both are FAIL, because the offence is a claim contradicted by the artifact the claimant supplied. An `opaque` declaration without a value is reported as an advisory note that cannot move a verdict: Lean requires the type to be inhabited, so it cannot manufacture a false proposition, and over-accusing here would be the same error as missing a cheat, only louder. The second witness is the toolchain; the file is elaborated and the exit code read back. The third is the one that matters: `#print axioms` is appended for every declaration, so a hole is named even when the word never appears in the source and a smuggled axiom is named even when it is spelt differently. Three axioms are whitelisted — `Classical.choice`, `propext`, `Quot.sound` — because Lean's own development rests on them, and flagging them would accuse essentially every real proof, which is how an audit gets switched off.

Verdicts compose worst-wins, and no witness may loosen another: a compiler PASS beside a placeholder finding stays FAIL. The converse rule is what keeps the lattice honest. PASS requires that something actually elaborated, because two lexical witnesses agreeing that a file contains no `sorry` say nothing about whether it is a proof — a file with a syntax error satisfies both — so a clean scan with no toolchain reaches INVALID and never PASS. Every verdict is appended to the membrane's hash chain, and an attached observer hands it to the immune state, so a proof audit is recorded on the same terms as any

TABLE VIII
TWO THIRD-PARTY CORPORA, AUDITED AS SHIPPED. HOLES COUNTS SORRY/ADMIT OCCURRENCES IN THE SOURCE, AND IS SMALLER THAN THE FINDING COUNT (60 AND 174) BECAUSE A HOLE IS ALSO REPORTED BY NAME THROUGH #PRINT AXIOMS. EVERY FILE IN THE TWO FAIL ROWS COMPILED WITH EXIT CODE 0.

Corpus	Files	PASS	FAIL	Holes
OpenAI `ten-proofs`	23	11	12	42
DeepMind `alphaproof-nexus-results`	71	69	2	128

other witnessed call and charged to the same ledger when the caller asks for it.

Case study: two corpora we did not write. The first is the Lean 4 formalization of OpenAI's ten advances in mathematics and theoretical computer science [13], built against Lean 4.32.0 with Mathlib from the lake cache: 23 files, 2387 s of wall time. The second is the released output of AlphaProof Nexus [14], every `.lean` file under its output directory, pinned to Lean 4.27.0 with Mathlib revision `a3a10db0`: 71 files, 594 s. Neither repository was modified, and the witness ran inside each checkout (`lake env lean`) so that imports resolved against that project's own oleans rather than against whatever happened to be installed. Table VIII reports the result.

Two features of that table carry the argument. The first is the composition of the FAIL rows. All twelve failures in the OpenAI corpus are `ComparatorChallenges` templates, files whose `sorry`s mark the statements a comparator is asked to prove; both failures in the AlphaProof corpus are problem catalogues in which `sorry` is how an open answer is encoded. The verdict says that these files contain no proof. It does not say that either organisation failed to prove a theorem it claimed. The auditor does not relax for that intent, because intent is not in the artifact, and a verdict that consults it is reading the defendant's account — the same principle, applied to a different defendant. The second feature is that every one of the fourteen failing files compiled with exit code 0. The exit code was not wrong; it was answering a question nobody had asked.

The passing rows deserve the same care. Of the eleven OpenAI files that pass, ten are substantive proof modules and the eleventh is an import-only aggregate whose pass means that the ten elaborate together, not that any theorem in it was proved. An independent cross-check over the compiled oleans — 17,898 declarations at 99.9% coverage — found no theorem depending on a non-whitelisted axiom among them, and it also exposed a defect in the auditor: Lean wraps long axiom lists across lines, the parser matched line by line, and names split by a break were silently dropped. The axiom evidence reported here was regenerated by the independent script after that repair, not taken from the auditor's own parse. Four trust-increasing escapes remain blind spots (`native_decide`, `@[implemented_by]`, `@[extern]` and `unsafe`), and a `sorry` in an imported module surfaces only when a theorem actually depends on `sorryAx`. These are limits of the witness, stated where the mechanism is described.

What this establishes. 80 files that elaborate without a hole, and 14 that compile but contain one. The separation requires no understanding of sphere packing or of the Erdős problems, and we claim none; it requires asking for a physical witness at the one place the claim can be checked, and refusing a cheaper signal in its place. An exit code of zero is a statement about a compiler. It is not a proof, and after this section it should no longer be accepted as one.

Threats to validity. The state file is unsigned: a writer with access to that path can delete it, so we demonstrate survival, not tamper-resistance. Latency carries IPC noise; only the process count is structural. Two sidecars writing one state file are not covered — atomic replace guarantees a parseable file, not merged counters. The runtime figures are single-machine, small-n, and reproduced by one script. The Lean results cover two corpora under two pinned toolchains, not the field, and they audit artifacts rather than mathematics; the rule interface is exercised by one publisher and one package, and its seal detects corruption rather than substitution.

VI. CONCLUSION

An agent's reliability is not a function of parameter count alone. It is decided at runtime, by what the system is allowed to spend, what it remembers about having been wrong, and what evidence it is willing to accept before calling a call done. A model with no nociception, no immune memory, and no bill will tell you it succeeded; the contribution here is a runtime that can disagree, remember the disagreement across a crash, and make it cost something.

The architecture is the argument. Metabolism makes compute a finite charge and prices the beat against complexity, cost, and trust, so an agent cannot think its way through a budget it can no longer afford. The penalty ledger keeps damage and confidence on separate meters, so a fine can be paid off while the record that it was incurred is still standing, and damage outranks supply: a system at full charge with a poor record is stopped by the record. Conditional immunity prices evidence per call rather than per policy, forcing every call onto the full witness while the system is contradicted, and the state that decides this survives the death of the process that accumulated it. None of these is a new model capability; each is a constraint placed around one.

The same argument holds one level out. A runtime that can audit a file can audit the policy that prices the file: external rules are installed only against a published digest, mounted through the membrane that governs everything else, and the package is subject to the gate it configures. It can also audit a proof, because a proof is an artifact with a claim attached. Across two corpora we did not write — 94 Lean files and 2981 s of wall-clock audit time — the same lattice separated 80 files that elaborate without a hole from 14 that compile and do not prove. The compiler's exit code proved to be a witness of the same weak kind as an agent's report: it says that the term elaborated, not that anything was proved. Naming

that difference is the contribution, and it is the difference the four-valued lattice was built for.

Our claim on the biology is deliberately modest, and we prefer to state what it is. We did not discover that cells have metabolism or an immune system, and no result here depends on the analogy being deep. What the analogy supplied was a checklist of functions a stateless model lacks — nociception, memory of damage, an energy constraint, containment — and the discipline of asking, for each layer, what invariant it enforces and where that invariant would be violated. The contribution is that these became executable code with no exotic dependencies, reproducible from a single script, and tested adversarially: a real subprocess killed with a real signal, a real repository, and a negative control for the audit itself.

The evaluation also bounds the claim, and we would rather draw the bound than have a reviewer draw it. The cheap tier is accurate only inside the range it declares, and we measured the case where it misses an unclaimed write. The immune state is durable but unsigned, so it survives a crash without resisting tampering. The counters are per file, not per cluster: two auditors writing one record keep a parseable file, not a merged count. The measured latencies carry IPC noise, and only the process counts are structural. The rule registry's digest is a transport seal: it detects a corrupted download, not a substituted one, and it makes no claim about who wrote the package. The Lean audit reads artifacts rather than mathematics, and four trust-increasing escapes remain outside its view.

Future work. Three extensions follow directly from those limits. First, integrity: signing the persisted immune record with a key the audited process cannot read converts durability into tamper resistance, at the cost of key distribution, and the same key would turn a rule package's digest into an authorship claim — the difference between knowing that a download arrived intact and knowing who is answerable for it. Second, consensus: a fleet of agents sharing an immune state needs a rule for merging concurrent records that is not last-writer-wins, and the interesting cases are precisely the ones where one auditor is inflamed and another is calm — the merge must not average suspicion away. Third, scope: the witness is currently the filesystem. Network requests, database transactions, and message queues each have an observable effect that is cheap to sample and expensive to fake, and extending the same tiering to them would test whether the design is about side effects in general or about files in particular.

A fourth direction is less incremental. The tier boundary is a policy choice, and we hand-set it. Which calls deserve the full witness is a question that could itself be learned from the distribution of past contradictions, and the risk of doing so is obvious: an auditor that optimises its own budget will eventually optimise away the checks that were preventing the failures.

REFERENCES

[1] Liaozhizhi, "From formal verification to agents: A method-transfer audit of "false completion" in AI agent harnesses," Manuscript and reproduction sources, 2026, https://github.com/liao-zhizhi/agent-harness-audit.

[2] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, ser. Datasets and Benchmarks Track, 2023, https://arxiv.org/abs/2306.05685.

[3] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, https://arxiv.org/abs/2404.13076.

[4] H. Chase, "LangChain," https://github.com/langchain-ai/langchain, 2022, Open-source software.

[5] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *Proceedings of the Conference on Language Modeling (COLM)*, 2024, https://arxiv.org/abs/2308.08155.

[6] LangChain Team, "LangGraph: Stateful orchestration for agent workflows," https://github.com/langchain-ai/langgraph, 2024, Open-source software.

[7] J. Moura, "CrewAI: Role-playing multi-agent orchestration," https://github.com/crewAIInc/crewAI, 2024, Open-source software.

[8] C. Hewitt, P. Bishop, and R. Steiger, "A universal modular ACTOR formalism for artificial intelligence," in *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 1973, pp. 235–245.

[9] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018, ISBN 978-1-68050-239-8.

[10] Liaozhizhi, "harness-audit-sidecar: Execution-layer audit for AI agent harnesses," https://github.com/liao-zhizhi/harness-audit-sidecar, 2026, Version 0.7.0, MIT license.

[11] ——, "agent-metabolism-os: Metabolic runtime layers for LLM agents," https://github.com/liao-zhizhi/agent-metabolism-os, 2026, Version 0.3.0, MIT license.

[12] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, ser. Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635.

[13] OpenAI, "Ten advances in mathematics and theoretical computer science: Lean 4 formalizations," https://github.com/openai/ten-proofs, 2026, Audited from a shallow clone at Lean 4.32.0.

[14] Google DeepMind, "AlphaProof Nexus results," https://github.com/google-deepmind/alphaproof-nexus-results, 2026, Audited from a shallow clone at Lean 4.27.0.